

1. Creating and Displaying a Vector

Question:

Write an R program to create a vector containing the marks of **10 students**. Display the vector and print the total number of elements in the vector.

Concepts Covered: `c()`, `length()`

```
mark = c(90,80,99,75,85,95,100,75,67,90)
```

```
print(mark)
```

```
print(length(mark))
```

```
> mark = c(90,80,99,75,85,95,100,75,67,90)
> print(mark)
[1] 90 80 99 75 85 95 100 75 67 90
> print(length(mark))
[1] 10
```

2. Arithmetic Operations on Vectors

Question:

Write an R program to create two vectors containing the monthly sales of two different stores. Perform the following operations:

- Addition
 - Subtraction
 - Multiplication
 - Division
- Display the results.

Concepts Covered: Vector arithmetic (+, -, *, /)

```
u = c(1,2,3,4,5)
```

```
v = c(5,6,7,8,9)
```

```
u + v
```

```
u - v
```

```
u * v
```

```
u/v
```

```

> u = c(1,2,3,4,5)
> v = c(5,6,7,8,9)
>
> u + v
[1] 6 8 10 12 14
> u - v
[1] -4 -4 -4 -4 -4
> u * v
[1] 5 12 21 32 45
> u/v
[1] 0.2000000 0.3333333 0.4285714 0.5000000 0.5555556

```

3. Finding Maximum, Minimum, Sum, and Average

Question:

Write an R program to create a vector of **8 employee salaries**. Find and display:

- Maximum salary
- Minimum salary
- Total salary
- Average salary

Concepts Covered: max(), min(), sum(), mean()

```
salary <- c(25000, 30000, 28000, 35000, 40000, 27000, 32000, 38000)
```

```
print(salary)
```

```
print(paste("Maximum Salary =", max(salary)))
```

```
print(paste("Minimum Salary =", min(salary)))
```

```
print(paste("Total Salary =", sum(salary)))
```

```
print(paste("Average Salary =", mean(salary)))
```

```

> salary <- c(25000, 30000, 28000, 35000, 40000, 27000, 32000, 38000)
>
> print(salary)
[1] 25000 30000 28000 35000 40000 27000 32000 38000
>
> print(paste("Maximum Salary =", max(salary)))
[1] "Maximum Salary = 40000"
> print(paste("Minimum Salary =", min(salary)))
[1] "Minimum Salary = 25000"
> print(paste("Total Salary =", sum(salary)))
[1] "Total Salary = 255000"
> print(paste("Average Salary =", mean(salary)))
[1] "Average Salary = 31875"

```

4. Sorting Employee Ages

Question:

Write an R program to create a vector containing the ages of **12 employees**. Display the vector in:

- Ascending order
- Descending order

Concepts Covered: sort(), decreasing = TRUE

```
age <- c(25, 30, 22, 28, 35, 40, 27, 24, 31, 29, 26, 33)
print(age)
print("Ascending Order")
print(sort(age))
print("Descending Order")
print(sort(age, decreasing = TRUE))
```

```
> age <- c(25, 30, 22, 28, 35, 40, 27, 24, 31, 29, 26, 33)
>
> print(age)
[1] 25 30 22 28 35 40 27 24 31 29 26 33
>
> print("Ascending Order")
[1] "Ascending Order"
> print(sort(age))
[1] 22 24 25 26 27 28 29 30 31 33 35 40
>
> print("Descending Order")
[1] "Descending Order"
> print(sort(age, decreasing = TRUE))
[1] 40 35 33 31 30 29 28 27 26 25 24 22
```

5. Searching for an Element

Question:

Write an R program to create a vector of **15 product IDs**. Read a product ID from the user and check whether it exists in the vector. Display an appropriate message.

Concepts Covered: %in%, which()

```
pid <- c(101,102,103,104,105,106,107,108,109,110,111,112,113,114,115)
id <- as.integer(readline("Enter Product ID: "))
if(id %in% pid)
{
  print("Product ID Found")
  print(which(pid == id))
}else
{
```

```

    print("Product ID Not Found")
  }

> id <- as.integer(readline("Enter Product ID: "))
Enter Product ID: 115
> if(id %in% pid){
+   print("Product ID Found")
+   print(which(pid == id))
+ }else{
+   print("Product ID Not Found")
+ }
[1] "Product ID Found"
[1] 15

```

6. Extracting Even and Odd Numbers

Question:

Write an R program to create a vector containing **20 integers**. Separate and display:

- Even numbers
- Odd numbers

Concepts Covered: Logical indexing, modulus operator (%%)

```

num <- c(1:20)

print("Even Numbers")

print(num[num %% 2 == 0])

print("Odd Numbers")

print(num[num %% 2 != 0])

> num <- c(1:20)
>
> print("Even Numbers")
[1] "Even Numbers"
> print(num[num %% 2 == 0])
[1]  2  4  6  8 10 12 14 16 18 20
>
> print("Odd Numbers")
[1] "Odd Numbers"
> print(num[num %% 2 != 0])
[1]  1  3  5  7  9 11 13 15 17 19
> |

```

7. Updating Vector Elements

Question:

Write an R program to create a vector containing the prices of **10 products**. Increase each product price by **10%** and display both the original and updated price vectors.

Concepts Covered: Vector arithmetic, assignment

```
price <- c(100,200,300,400,500,600,700,800,900,1000)
```

```
print("Original Prices")
```

```
print(price)
```

```
updated <- price * 1.10
```

```
print("Updated Prices")
```

```
print(updated)
```

```
> price <- c(100,200,300,400,500,600,700,800,900,1000)
>
> print("Original Prices")
[1] "Original Prices"
> print(price)
[1] 100 200 300 400 500 600 700 800 900 1000
>
> updated <- price * 1.10
>
> print("Updated Prices")
[1] "Updated Prices"
> print(updated)
[1] 110 220 330 440 550 660 770 880 990 1100
```

8. Vector Indexing and Slicing

Question:

Write an R program to create a vector of **15 city names**. Display:

- First five cities
- Last five cities
- Cities from index 6 to 10
- Every alternate city

Concepts Covered: Indexing, slicing, sequence operator (:), seq()

```
city <- c("Chennai","Delhi","Mumbai","Bangalore","Hyderabad",
```

```
        "Pune","Kolkata","Jaipur","Lucknow","Goa",
```

```
        "Mysore","Cochin","Madurai","Surat","Nagpur")
```

```
print("First Five Cities")
```

```
print(city[1:5])
```

```
print("Last Five Cities")
```

```

print(city[11:15])
print("Cities from Index 6 to 10")
print(city[6:10])
print("Alternate Cities")
print(city[seq(1,15,2)])

> city <- c("Chennai","Delhi","Mumbai","Bangalore","Hyderabad",
+          "Pune","Kolkata","Jaipur","Lucknow","Goa",
+          "Mysore","Cochin","Madurai","Surat","Nagpur")
>
> print("First Five Cities")
[1] "First Five Cities"
> print(city[1:5])
[1] "Chennai" "Delhi" "Mumbai" "Bangalore" "Hyderabad"
>
> print("Last Five Cities")
[1] "Last Five Cities"
> print(city[11:15])
[1] "Mysore" "Cochin" "Madurai" "Surat" "Nagpur"
>
> print("Cities from Index 6 to 10")
[1] "Cities from Index 6 to 10"
> print(city[6:10])
[1] "Pune" "Kolkata" "Jaipur" "Lucknow" "Goa"
>
> print("Alternate Cities")
[1] "Alternate Cities"
> print(city[seq(1,15,2)])
[1] "Chennai" "Mumbai" "Hyderabad" "Kolkata" "Lucknow" "Mysore" "Madurai"
[8] "Nagpur"

```

9. Statistical Analysis of Student Marks

Question:

Write an R program to create a vector containing the marks of **20 students**. Calculate and display:

- Mean
- Median
- Standard deviation
- Variance
- Range

Concepts Covered: mean(), median(), sd(), var(), range()

```

marks <- c(78,85,90,67,88,92,75,80,69,95,
           84,77,73,81,89,91,66,79,87,94)

```

```

print("Mean")
print(mean(marks))
print("Median")

```

```

print(median(marks))

print("Standard Deviation")

print(sd(marks))

print("Variance")

print(var(marks))


print("Range")

print(range(marks))

> marks <- c(78,85,90,67,88,92,75,80,69,95,
+           84,77,73,81,89,91,66,79,87,94)
>
> print("Mean")
[1] "Mean"
> print(mean(marks))
[1] 82
>
> print("Median")
[1] "Median"
> print(median(marks))
[1] 82.5
>
> print("Standard Deviation")
[1] "Standard Deviation"
> print(sd(marks))
[1] 8.932495
>
> print("Variance")
[1] "Variance"
> print(var(marks))
[1] 79.78947
>
> print("Range")
[1] "Range"
> print(range(marks))
[1] 66 95

```

10. Employee Performance Analysis

Question:

Write an R program to create a vector containing the performance scores of **15 employees**. Perform the following operations:

- Display employees scoring above 80.
- Count the number of employees scoring below 50.
- Find the highest and lowest scores.
- Calculate the average performance score.
- Display the scores in descending order.

Concepts Covered: Logical indexing, sum(), max(), min(), mean(), sort()

```
score <- c(45,82,67,90,55,78,88,49,95,60,72,81,39,86,93)
print("Scores Above 80")
print(score[score > 80])
print("Count Below 50")
print(sum(score < 50))
print("Highest Score")
print(max(score))
print("Lowest Score")
print(min(score))
```

```
print("Average Score")
print(mean(score))
print("Descending Order")
print(sort(score, decreasing = TRUE))
```

```
> score <- c(45,82,67,90,55,78,88,49,95,60,72,81,39,86,93)
>
> print("Scores Above 80")
[1] "Scores Above 80"
> print(score[score > 80])
[1] 82 90 88 95 81 86 93
>
> print("Count Below 50")
[1] "Count Below 50"
> print(sum(score < 50))
[1] 3
>
> print("Highest Score")
[1] "Highest Score"
> print(max(score))
[1] 95
>
> print("Lowest Score")
[1] "Lowest Score"
> print(min(score))
[1] 39
>
> print("Average Score")
[1] "Average Score"
> print(mean(score))
[1] 72
>
> print("Descending Order")
[1] "Descending Order"
> print(sort(score, decreasing = TRUE))
[1] 95 93 90 88 86 82 81 78 72 67 60 55 49 45 39
```


